

Beginner's Deployment Guide

Never deployed to the cloud before? Follow 13 numbered steps, each with what to type, what you should see, and what to do if it goes wrong. You'll put StreamFlix on the internet and then safely delete it all again.

React 19

Node.js 22

6 microservices

PostgreSQL 16

Redis 7

Docker

Kubernetes / EKS

Helm

Terraform

Jenkins

Prometheus

Grafana

Audience

First-time deployers

Target

AWS EKS · us-east-1 (N. Virginia)

Date

3 October 2026

Est. time

~2 hours start to finish

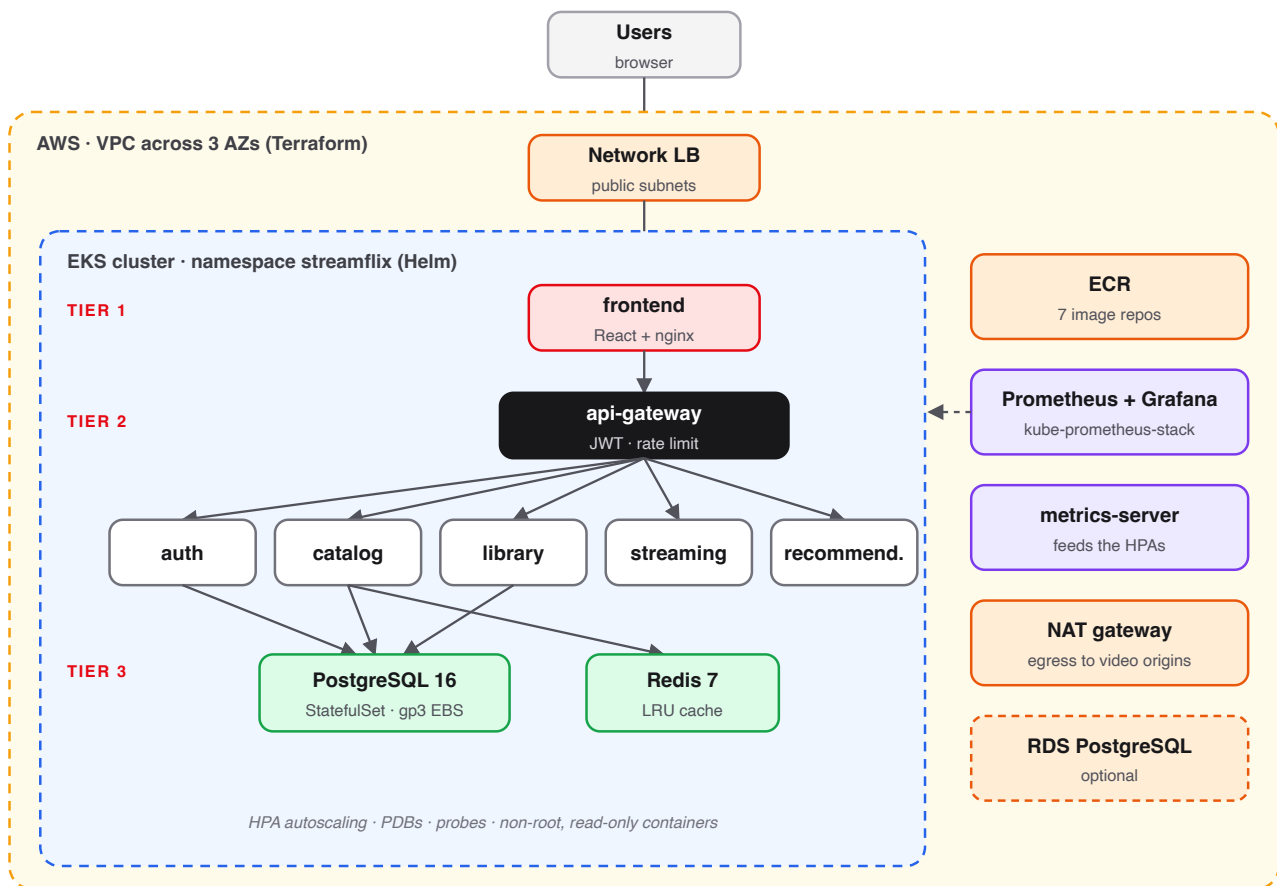
Est. cost

≈ \$0.30 / hour · \$1–2 per session

Source

github.com/sanjaybdch9/streamflix

Architecture at a glance



Your journey: 13 steps



Four rules for a smooth first deployment

1 · Go in order

Each step depends on the one before. Use the checklist in Part E and tick each step off.

2 · Set the budget alert first

Step 2 emails you before costs get big. AWS charges about \$0.30 an hour while StreamFlix runs.

3 · Never share your secret key

Not in chat, email, GitHub or screenshots. If it leaks, delete it in IAM straight away.

4 · Always finish with Step 13

Delete in the order given, then check the AWS console. Nothing left running means nothing charged.

You have never deployed anything to the cloud before. That's fine. This guide takes you from zero to StreamFlix running on the internet with its own public web address, one small step at a time.

Every step has the same four parts:

- **Why:** what the step does, in plain words.
- **Do:** exactly what to click or type.
- **You should see:** how to know it worked.
- **If it goes wrong:** the usual fix.

 **How to use this guide.** Do the steps in order and don't skip ahead. Each step depends on the one before. Tick each box in the checklist at the end as you go. If you get stuck, copy the exact error message and ask for help; the error text tells you almost everything.

Part A — Understand what you're building (10 minutes of reading)

The big picture

StreamFlix is a Netflix-style website. You'll put it on **Amazon Web Services (AWS)**, which rents out computers in its data centres by the hour. When you finish, anyone on the internet can open StreamFlix in a browser.

The app has **three tiers** (layers):

Tier	What it is	Real-world comparison
1. Presentation	The website you see (React)	The shop window
2. Application	6 small programs ("microservices") that do the work: login, catalog, video, etc.	The staff behind the counter
3. Data	PostgreSQL database + Redis cache	The storeroom and filing cabinets

The words you'll meet

Word	Meaning in one line
AWS	Amazon's cloud. You rent computers, storage and networking by the hour.
Region	Which AWS data-centre location you use. We use us-east-1 (North Virginia, USA).
IAM user	A login for AWS with specific permissions. You'll create one for yourself.
Access key	A username/password pair for <i>programs</i> (not people) to use AWS. Keep it secret.
Container	A program packed with everything it needs, so it runs the same everywhere.
Image	The packed, ready-to-run version of a container, like an installer file.

Word	Meaning in one line
Docker	The tool that builds images and runs containers.
ECR	AWS's storage for your images (a private "app store" for your containers).
Kubernetes	Software that runs many containers across many computers and keeps them healthy.
EKS	AWS's managed Kubernetes. AWS runs the hard parts for you.
Node	One computer (virtual machine) in the Kubernetes cluster. You'll have 3.
Pod	One running copy of a container inside Kubernetes.
kubectl	The command you type to talk to Kubernetes ("kube-control").
Helm	Installs a whole app into Kubernetes in one command, from a "chart" (a recipe).
Terraform	Creates cloud infrastructure from code files, so you don't click through the AWS console.
VPC	Your own private network inside AWS.
Load balancer	The public front door that gives your app an internet address.
Terminal	The app where you type commands. On Mac: press ⌘ Space, type <i>Terminal</i> , press Enter.

What the deployment does, in order

```

Your Mac —(Terraform)—> AWS builds: network + Kubernetes cluster + image storage
Your Mac —(Docker)—> builds 7 images —> uploads them to AWS (ECR)
Your Mac —(Helm)—> tells Kubernetes to run StreamFlix from those images
AWS —> gives you a public web address —> you open StreamFlix

```

Part B — Before you start

⚠ This costs real money

AWS charges by the hour while things are running:

Item	Approx. cost
EKS cluster (Kubernetes control plane)	\$0.10 / hour
3 × t3.medium computers (nodes)	\$0.12 / hour
NAT gateway (lets the cluster reach the internet)	\$0.045 / hour + data
Load balancer	\$0.02 / hour
Total	≈ \$0.30 / hour ≈ \$7 / day ≈ \$210 / month

These figures are approximate. They're **not covered by the AWS free tier**. The plan: deploy, explore for a few hours, then **delete everything (Step 13)**. A 3-hour session costs about **\$1–2**.

Time needed

Part	Time
AWS account and safety setup (Steps 1–3)	20–30 min (once ever)
Mac setup (Steps 4–6)	10 min (mostly already done on your Mac )
Local test (Step 7)	10 min
Deploy to AWS (Steps 8–10)	~45 min, mostly waiting
Explore (Steps 11–12)	as long as you like
Delete everything (Step 13)	15 min

What you need

- A Mac with internet access (you have this .
- A credit or debit card for the AWS account.
- A phone with an authenticator app (Google Authenticator, Microsoft Authenticator, or similar) for account security.

Part C — The steps

Step 1 — Create an AWS account (skip if you have one)

Why: you need an account to rent AWS resources.

Do:

1. Go to <https://aws.amazon.com> and click **Create an AWS Account**.
2. Enter your email and an account name (e.g. `sanjay-learning`), verify the email, and set a strong password.
This first login is called the **root user**; it controls everything.
3. Choose **Personal** account, fill in your details, and add your card.
4. Verify your phone number.
5. Choose the **Basic support – Free** plan.

Then secure the root user (important):

6. Sign in at <https://console.aws.amazon.com> as the root user.
7. Click your account name (top-right) → **Security credentials** → **Assign MFA device**.
8. Choose **Authenticator app**, scan the QR code with your phone app, and type in two codes.

You should see: "MFA device assigned".

 After today, you'll almost never use the root user. You'll use the IAM user from Step 3.

Step 2 — Set a spending alert (do not skip)

Why: if you forget to delete things, AWS keeps charging. A budget emails you before the bill gets big.

Do:

1. In the AWS console, type **Budgets** in the top search bar and open **Billing and Cost Management** → **Budgets**.
2. Click **Create budget** → **Use a template** → **Monthly cost budget**.
3. Budget name: `streamflix-limit`. Budgeted amount: `20` (USD).
4. Email recipients: your email address. Click **Create budget**.

You should see: the budget listed. AWS emails you when you reach 85% and 100% of \$20.

Step 3 — Create your IAM user and access key

Why: never use the root user for daily work. You create a separate user, and give your Mac an **access key** so Terraform and the other tools can act on your behalf.

Do — create the user:

1. Console search bar → **IAM** → left menu **Users** → **Create user**.
2. User name: `devops-admin` → **Next**.
3. Choose **Attach policies directly**. Search for `AdministratorAccess` and tick it → **Next** → **Create user**.

⚠ AdministratorAccess is fine for your personal learning account. At a company, you'd get narrower permissions from your platform team.

Do — create the access key:

4. Click the new user `devops-admin` → the **Security credentials** tab.
5. Scroll to **Access keys** → **Create access key**.
6. Choose **Command Line Interface (CLI)**, tick the confirmation box → **Next** → **Create access key**.
7. Click **Download .csv file** and keep it somewhere safe.

You should see: an **Access key ID** (starts with `AKIA...`) and a **Secret access key** (long random text).

🔒 **Treat the secret key like a bank password.** Never paste it into chat, email, GitHub or screenshots. Anyone who has it can spend money on your account. If it ever leaks, delete it in the IAM console straight away and create a new one.

Step 4 — Install the tools on your Mac ✅ (already done)

Why: these are the programs that talk to AWS, Docker and Kubernetes.

On your Mac, all of them are already installed. To check, open Terminal and run:

```
docker --version && aws --version && terraform version && helm version && kubectl version --client
```

You should see: five version lines, with no `command not found`.

If something is missing, install it with Homebrew:

```
brew install awscli hashicorp/tap/terraform helm kubectl && brew install --cask docker-desktop
```

Also make sure Docker Desktop is running: you should see the whale icon in the menu bar at the top of your screen. If not, open **Docker** from Applications and wait until it says **Engine running**.

Step 5 — Connect your Mac to AWS

Why: this saves your access key on your Mac so the tools can use your AWS account.

Do: in Terminal, run:

```
aws configure
```

It asks four questions. Type each answer and press Enter:

Question	What to type
AWS Access Key ID	The <code>AKIA...</code> value from your .csv file
AWS Secret Access Key	The secret from your .csv (it won't show while you type; that's normal)
Default region name	<code>us-east-1</code>
Default output format	<code>json</code>

Check it worked:

```
aws sts get-caller-identity
```

You should see something like this, with your own numbers:

```
{
  "UserId": "AIDA...",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/devops-admin"
}
```

Write down your 12-digit **Account** number; you'll see it again later.

If it goes wrong:

Error	Fix
<code>Unable to locate credentials</code>	Run <code>aws configure</code> again and re-enter the keys
<code>InvalidClientTokenId</code> or <code>SignatureDoesNotMatch</code>	A key was mistyped. Run <code>aws configure</code> again and copy from the .csv carefully

Step 6 — Open the project folder

Why: every command from now on must be run from inside the StreamFlix project folder.

Do:

```
cd ~/Documents/streamflix && git pull && ls
```

You should see files including `README.md`, `docker-compose.yml`, `terraform`, `helm`, `scripts`, `services` and `frontend`.

💡 If you open a **new** Terminal window later, run `cd ~/Documents/streamflix` again first. Most "No such file or directory" errors come from being in the wrong folder.

(On a different computer, download the project first: `git clone https://github.com/sanjaybdch9/streamflix.git && cd streamflix`.)

Step 7 — Test on your Mac first (free)

Why: check the app works on your own computer before you pay for AWS. If it fails here, it will fail in AWS too.

Do:

```
cp -n .env.example .env; docker compose up --build -d
```

(`cp -n` won't overwrite your existing `.env`. If this is a fresh download, open `.env` and replace each `change-me` value with random text first.)

Wait about a minute, then run:

```
curl -s localhost:8080/api/status
```

You should see: `"status":"ok"`, with every service `"up"`.

Now open <http://localhost:8080> in your browser, click **Sign up now**, create an account and play a video. 🎉

Then stop it, to free up your Mac's memory for the next steps:

```
docker compose down
```

If it goes wrong:

Problem	Fix
Cannot connect to the Docker daemon	Open Docker Desktop and wait for "Engine running"
"status":"degraded"	Wait 30 more seconds and try again; the database starts slowly the first time

Problem	Fix
Browser says "Failed to fetch"	The backend isn't running. Run <code>docker compose ps</code> , and every row should say "running"

Step 8 — Build the AWS infrastructure with Terraform (~20 minutes)

Why: Terraform reads the files in the `terraform/` folder and creates, in your AWS account: a private network (VPC), a Kubernetes cluster (EKS) with 3 computers, and 7 image repositories (ECR).

Do — 8a. Go into the terraform folder and prepare the settings file:

```
cd ~/Documents/streamflix/terraform && cp terraform.tfvars.example terraform.tfvars
```

The defaults are fine for a beginner (region `us-east-1`, 3 × t3.medium computers). You don't need to edit anything.

Do — 8b. Download Terraform's plugins (once):

```
terraform init
```

You should see: a green "Terraform has been successfully initialized!"

Do — 8c. Preview what will be created (nothing is created or charged yet):

```
terraform plan
```

You should see, at the bottom, something like: `Plan: NN to add, 0 to change, 0 to destroy.` where NN is a number around 60–80.

✅ The important part is `0 to destroy`. You're only adding new things.

Do — 8d. Create everything (this starts the charges):

```
terraform apply
```

Terraform shows the plan again and asks "Do you want to perform these actions?" Type `yes` and press Enter.

Now **wait 15–20 minutes**. You'll see lines like `module.eks...: Still creating... [10m0s elapsed]`. This is normal; creating the cluster is the slow part. Don't close the Terminal window.

You should see, at the end: `Apply complete! Resources: NN added, 0 changed, 0 destroyed.` followed by `Outputs:` showing `cluster_name = "streamflix"`, your `ecr_registry`, and a few more.

Do — 8e. Connect kubectl to your new cluster and check it:

```
aws eks update-kubeconfig --region us-east-1 --name streamflix && kubectl get nodes
```

You should see: 3 lines, each with **Ready** in the STATUS column.

Go back to the main project folder:

```
cd ~/Documents/streamflix
```

⚠ **Never delete the `terraform` folder**, and especially not the `terraform.tfstate` file in it. That file is Terraform's memory of what it created. Without it, Step 13 can't delete everything for you.

If it goes wrong:

Error	Fix
<code>No valid credential sources found</code>	Step 5 didn't work. Run <code>aws sts get-caller-identity</code>
<code>AccessDenied</code> / <code>UnauthorizedOperation</code>	Your IAM user lacks AdministratorAccess (Step 3)
<code>VcpuLimitExceeded</code> or vCPU quota errors	New accounts sometimes have low limits. In the console: Service Quotas → EC2 → Running On-Demand Standard instances , request 16 or more, wait for approval, then run <code>terraform apply</code> again
It stopped halfway with an error	Fix the cause, then run <code>terraform apply</code> again. It continues where it stopped

Step 9 — Install the cluster helpers (~5 minutes)

Why: two helpers are needed before the app goes in:

- **metrics-server** measures CPU use, so Kubernetes can add more copies of a service when it's busy.
- **Prometheus + Grafana** collect and show the app's health charts.

Do:

```
scripts/eks-bootstrap.sh us-east-1 streamflix
```

You should see at the end: `Cluster add-ons installed.` and a **Grafana password**. **Copy that password into your notes now**; you'll need it in Step 11.

Check it:

```
kubectl get pods -n monitoring
```

Every line should say `Running`. (If some say `ContainerCreating`, wait a minute and check again.)

Step 10 — Deploy StreamFlix (~10 minutes)

Why: this one script does the whole app deployment:

0. **Safety check:** it refuses to deploy unless every change is committed and pushed to GitHub. That way, what runs on AWS always matches your GitHub repository.

1. Builds the 7 app images on your Mac. They're built for Intel/AMD (`linux/amd64`), the chip type the AWS computers use, even though your Mac has an Apple chip.
2. Uploads them to your private image storage (ECR).
3. Tells Kubernetes, through Helm, to run them, and waits until every part is healthy.
4. Runs a quick health test and prints your app's public web address.

Do: make sure Docker Desktop is running, then:

```
scripts/deploy.sh us-east-1 streamflix
```

If it stops with **Deploy blocked**, it tells you exactly what to do, usually `git add -A && git commit -m "..."` && `git push`. Run that, then run the script again.

The first build takes 5–10 minutes. You'll see `==> Building api-gateway`, then the other services, then `==> Deploying with Helm`.

You should see at the end:

```
==> Smoke test
{"status":"ok","services":[{"name":"auth","status":"up",...}]}
==> Done. App: http://a1b2c3....elb.us-east-1.amazonaws.com
```

 **Open that `http://...amazonaws.com` address in your browser.** It may take **2–3 minutes** before it loads the first time, because AWS is setting up the address. Keep refreshing. Then sign up, browse, and play a video. **StreamFlix is live on the internet.**

Have a look at what's running:

```
kubectl -n streamflix get pods
```

You'll see 14 or more pods (copies of each service, plus the database and cache), all **Running**.

If it goes wrong:

Error	Fix
Cannot connect to the Docker daemon	Start Docker Desktop, then run the script again
Deploy blocked: you have uncommitted changes	Run <code>git add -A && git commit -m "your message" && git push</code> , then run the script again
Deploy blocked: commit ... is not on GitHub yet	Run <code>git push</code> , then run the script again
no matches for kind "ServiceMonitor"	You skipped Step 9. Run it, then run this step again
Script ends with UPGRADE FAILED / timed out	Run <code>kubectl -n streamflix get pods</code> , find the one that isn't Running , and run <code>kubectl -n streamflix logs <pod-name></code> . The last lines say why. Fix it and run the script again; Helm undoes failed attempts automatically

Error	Fix
Browser can't open the address	Wait 3 minutes (DNS is still being set up), and use <code>http://</code> , not <code>https://</code>
The page loads but shows "Failed to fetch"	Run <code>kubectl -n streamflix get pods</code> ; the <code>api-gateway</code> pods must be <code>Running</code>

Step 11 — Look at the health dashboard (Grafana)

Why: this is how DevOps engineers watch a live system: traffic, speed, errors and video streams.

Do:

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
```

This keeps running; leave that Terminal window open. Then in your browser:

1. Open `http://localhost:3000`.
2. Log in with user `admin` and the password from Step 9.
3. Go to **Dashboards** → **StreamFlix — Service Overview**.

Use StreamFlix in another tab (play videos, search) and watch the charts move.

To stop it, go back to that Terminal window and press **Control + C**.

Step 12 — Practise a change and a rollback (optional, recommended)

Why: in a real DevOps job, you deploy changes often, and sometimes need to undo one quickly.

Make a change:

1. Open `frontend/src/components/Hero.jsx` in any text editor.
2. Find `'FILM'` and change it to `'NOW SHOWING'`. Save.
3. Commit the change, push it to GitHub, and deploy it:

```
git add -A && git commit -m "Change hero label" && git push && scripts/deploy.sh us-east-1 streamflix
```

4. Refresh the website. The label above the big title has changed.

Undo it (roll back):

```
helm -n streamflix history streamflix
```

This lists your releases (revisions 1, 2, ...). Go back to revision 1:

```
helm -n streamflix rollback streamflix 1
```

Refresh the site after about a minute. The old label is back. That's a rollback.

Step 13 — DELETE EVERYTHING when you're done (do not skip)

Why: AWS keeps charging about \$7 a day until you delete it. Delete in **this exact order**. Kubernetes created a load balancer and a disk that Terraform doesn't know about, and if they're still there, Terraform can't delete the network.

Make sure you're in the project folder (`cd ~/Documents/streamflix`), then run one block at a time.

13a. Remove the app (this deletes the load balancer and database):

```
helm -n streamflix uninstall streamflix && kubectl -n streamflix delete pvc --all && kubectl delete namespace streamflix
```

13b. Remove the monitoring tools:

```
helm -n monitoring uninstall kube-prometheus-stack && kubectl delete namespace monitoring
```

13c. Delete the leftover database disk. It's kept on purpose, so data isn't lost by accident.

```
for v in $(aws ec2 describe-volumes --region us-east-1 --filters Name=tag:kubernetes.io/created-for/pvc/namespace,Values=streamflix --query 'Volumes[].VolumeId' --output text); do echo "deleting $v"; aws ec2 delete-volume --region us-east-1 --volume-id $v; done
```

If this says `VolumeInUse` , wait 1 minute and run it again.

13d. Wait 2 minutes for AWS to finish removing the load balancer. Then delete all the infrastructure:

```
cd terraform && terraform destroy
```

Type `yes` when asked. It takes about 10–15 minutes.

You should see: `Destroy complete! Resources: NN destroyed.` (the same NN as in Step 8)

13e. Double-check in the AWS console (region **N. Virginia / us-east-1**, top-right):

Console page	Should be
EKS → Clusters	empty
EC2 → Instances	none running (terminated ones disappear after a while)
EC2 → Load Balancers	empty
EC2 → Volumes	none left with "streamflix" or "pvc" in the name
VPC → NAT gateways	none <code>Available</code>

 When those are empty, nothing is costing you money. Your Step 2 budget email is the final safety net.

If **terraform destroy** gets stuck or fails (often on the VPC or subnets): a load balancer or network interface is still there. Go to **EC2 → Load Balancers** and delete any left, wait 2 minutes, then run **terraform destroy** again.

Part D — One-page cheat sheet

```
# ----- one-time -----
aws configure                                # Step 5
aws sts get-caller-identity                  # check AWS access

# ----- deploy -----
cd ~/Documents/streamflix
cd terraform && terraform init && terraform apply && cd ..      # Step 8 (~20 min)
scripts/eks-bootstrap.sh us-east-1 streamflix                 # Step 9 (~5 min)
scripts/deploy.sh us-east-1 streamflix                         # Step 10 (~10 min)

# ----- look around -----
kubectl -n streamflix get pods                # are all parts running?
kubectl -n streamflix get svc frontend         # the public address (EXTERNAL-IP)
kubectl -n streamflix logs deploy/api-gateway  # read one service's log
helm -n streamflix history streamflix          # list releases
helm -n streamflix rollback streamflix 1       # undo to revision 1

# ----- delete everything (Step 13, in this order) -----
helm -n streamflix uninstall streamflix
kubectl -n streamflix delete pvc --all && kubectl delete namespace streamflix
helm -n monitoring uninstall kube-prometheus-stack && kubectl delete namespace monitoring
# delete leftover volumes (Step 13c), wait 2 min, then:
cd terraform && terraform destroy
```

Part E — Your progress checklist

✓	Step	Done when you see
☐	1. AWS account + root MFA	"MFA device assigned"
☐	2. Budget alert	streamflix-limit budget listed
☐	3. IAM user + access key	.csv file downloaded and stored safely
☐	4. Tools installed	5 version numbers printed
☐	5. aws configure	your 12-digit account ID
☐	6. Project folder	ls shows terraform , helm , scripts
☐	7. Local test	StreamFlix plays at localhost:8080

✓	Step	Done when you see
☐	8. Terraform	<code>Apply complete!</code> and 3 nodes <code>Ready</code>
☐	9. Cluster helpers	Grafana password saved
☐	10. Deploy	public <code>...amazonaws.com</code> address works
☐	11. Grafana	StreamFlix dashboard shows charts
☐	12. Change + rollback	the label changed, then changed back
☐	13. Delete everything	<code>Destroy complete!</code> and the console checks are empty

Part F — What to learn next

- **Jenkins CI/CD:** let a server run Step 10 automatically on every `git push`. See `DEPLOYMENT.md`, Phase 5.
- **Real video hosting:** S3 + MediaConvert + CloudFront, the way real streaming companies do it.
- **HTTPS:** a custom domain with a free certificate from AWS Certificate Manager.
- **Cheaper practice clusters:** set `node_capacity_type = "SPOT"` in `terraform/terraform.tfvars` for about 70% lower compute cost.